

Equiripple Group-Delay IIR Digital Filters

Kenneth Martin
Granite SemiCom Inc.
Toronto ON, Canada
martin@granitesemi.com

Abstract—Filters having linear-phase frequency responses preserve pulse shapes and are often used in communication systems. These are typically realized as finite-impulse-response filters; only approximately linear-phase filters can be realized using infinite-impulse-response filters. Classical examples include Bessel-Thomson filters and equiripple group-delay (EQUI-GD) filters for continuous-time IIR filters. A popular technique for realizing IIR digital filters uses a bilinear-z transform. This technique has a one-to-one frequency mapping of magnitude response but does not preserve equiripple group-delay. This paper presents a method for correcting the group-delay of digital filters based on bilinear-z transforms. The resulting filters have highly accurate equiripple group-delay.

Index Terms—linear-phase filters, constant group-delay, equiripple group delay, digital filters with linear phase response

I. INTRODUCTION

This paper presents design techniques for approximately linear-phase IIR digital filters.

Two major techniques have been presented for designing continuous-time infinite-impulse-response (IIR) filters having almost linear-phase frequency response. One technique is based on having maximally flat group-delay response at the pass-band center. This technique is used for Bessel-Thomson filters and design routines are available [1]. A second technique results in filter transfer functions having group-delays that are almost flat in the pass-band, but with equiripple errors about an average value. Design equations for realizing these filters are available in difficult to find texts, but computer realizations of these equations can not be located by the author.

One purpose of this presentation is to describe Matlab routines that result in IIR filters having equiripple group delays in the pass-band. Routines are also described for realizing IIR transfer functions having maximally flat group-delay at the pass-band center frequency. These filters are used as a starting point for iterative techniques used in finding equiripple discrete-time transfer functions; this approach is new. Both types of filters can have stop-bands that have finite-transmission-zeros on the $j\omega$ axis. Matlab routines for this extension are also described. Thus, a major aim of the paper is to make available Matlab routines that can be used to design classical IIR filter transfer functions having almost linear-phase frequency responses. The routines are robust and can be used for orders greater than 11 for any reasonable specified pass-band edge loss. This feature is more flexible than what is available using tables.

The main goal of this paper is to show how *digital* IIR filters having almost linear-phase group-delay response can be designed. These digital filters can be realized using cascade structures; as in exemplified in an example.

A popular technique for realizing digital filters is based on applying a bilinear-z transform to continuous-time transfer functions. This technique compresses frequencies between pass-band edges and infinity, in the continuous-time domain, to frequencies between -0.5 and 0.5 times the sampling frequency, in the discrete-time domain. This approach preserves magnitude response characteristics except for compressing the stop-band frequencies. Unfortunately, for continuous-time filters having equiripple group-delay characteristics; the use of the bilinear-z transform approach seriously degrades the equiripple group-delay characteristic. This paper presents the details of an iterative approach used for changing pole positions that corrects for the degradations introduced by using the bilinear-z transform method. The resulting digital filters have highly-accurate equiripple group-delays in the pass-band (and up to the frequencies of the closest loss poles).

II. ALL-POLE IIR FILTERS HAVING MAXIMALLY FLAT GROUP-DELAY

Continuous-time transfer functions having maximally-flat group-delay at the pass-band center (which is 0 rad. for a low-pass filter) are attributed to W. E. Thomson from the British Post Office Research Station and are commonly called Bessel-Thomson filters. These filters are all-pole filters and the denominator polynomials can be found using the recursive equations (12-155) of [1].

$$E^n(s) = (2n - 1)E^{n-1}(s) + s^2 E^{n-2}(s) \quad (1)$$

Matlab code that returns Bessel polynomials is (with comments removed):

```
function H = BesselFltr(n) e2 = [1];  
e1 = [1 1];  
for i = 2:n  
    en = [e2 0 0] + (2*i - 1)*[0 e1];  
    e2 = e1;  
    e1 = en;  
end  
TF = tf([en(n+1)], en);  
H = zpkl(TF);
```

Our routines uses this approach and frequency-scaling to adjust the attenuation of a low-pass filter to be equal to the specified loss (in dB) at 1 rad. First the frequency of the low-pass prototype having the desired pass-band loss is found using

an iterative approach based on derivatives. Secondly, the low-pass prototype is frequency scaled so that its attenuation at the pass-band edge (which is not necessarily 1 rad.) is equal to the specified attenuation. Finally, the low-pass prototype is frequency shifted using a complex shift to any desired band-pass frequency. Before describing details of this methodology, a review of equations for finding frequency derivatives of continuous-time transfer functions is given.

Consider a transfer function expressed in pole-zero form.

$$H(s) = \frac{k \prod_{i=1}^{N_z} (s - z_i)}{\prod_{i=1}^{N_p} (s - p_i)} \quad (2)$$

We have

$$\ln(H(s)) = \ln(k) + \sum_{i=1}^{N_z} \ln(s - z_i) - \sum_{i=1}^{N_p} \ln(s - p_i) \quad (3)$$

Consider differentiating $\ln(H(s))$ with respect to frequency (ω)

$$\frac{\partial \ln(H(s))}{\partial w} = \sum_{i=1}^{N_z} \frac{j}{s - z_i} - \sum_{i=1}^{N_p} \frac{j}{s - p_i} \quad (4)$$

and the group-delay ($T(s)$) is given by

$$T(\omega) = \text{real}\left(\sum_{i=1}^{N_z} \frac{1}{s - z_i} - \sum_{i=1}^{N_p} \frac{1}{s - p_i}\right) \quad (5)$$

for $s = j\omega$. Equation (5) can be used to compute the group-delay and is easily differentiated with respect to the poles p_i to adapt to minimize the group-delay. Based on the above equations the following Matlab code can be used analyze a transfer function for its gain, group-delay, and derivatives.

```
[z, p, k] = zpkdata(H, 'vector');
npts = length(w);
onesVctr = ones(size(w));
np = length(p);
nz = length(z);
derivSum = zeros(size(w));
logSum = zeros(size(w));
ddrvSum = zeros(size(w));
s = j.*w;
for l = 1:nz
    logSum = logSum + log((s - z(l)));
    derivSum = derivSum + 1./(s - z(l)*onesVctr);
    ddrvSum = ddrvSum + 1./((s - z(l)*onesVctr).^2);
end
for l = 1:np
    logSum = logSum - log((s - p(l)));
    derivSum = derivSum - 1./(s - p(l)*onesVctr);
    ddrvSum = ddrvSum - 1./((s - p(l)*onesVctr).^2);
end
phH = imag(logSum);
lgH = log(k) + real(logSum);
gdH = -real(derivSum);
% -real instead of imag because we didn't
% multiply by j when finding first order derivatives
dLdW = -imag(derivSum);
dTdW = -imag(ddrvSum);
```

Using this function, the top level code for designing all-pole filters having maximally-flat group-delay is (with comments and error-checking removed):

```
function H = besselfilt(ni, wp, ap)
H1 = BesselFtr(ni);
w = 1;
ep = -ap*log(10)/20; % desired loss in nepers
for i = 1:10
    [lgH, phH, gdH, dLdW, dTdW] = AnlzH(H1, w);
    w = w - (lgH - ep)/dLdW;
end

H2 = scaleFtr(H1, 0.5*diff(wp)/w);
wShft = mean(wp);
H = freq_shift(H2, wShft);
```

III. EQUIRIPPLE GROUP DELAY FILTERS

The design of filters having equiripple group-delay can be achieved using the approach of [2] where the n non-linear equations of (2.75) on page 60 are solved using a non-linear equation solver. These equations are:

$$\pm \frac{(z_k^2 - 1)^{3/2}}{z_k^2} + \omega_p \Delta T \prod_{\substack{i=1 \\ i \neq k}}^n \frac{z_i + z_k}{z_i - z_k} = 0 \quad (6)$$

where the z_i are the transformed natural modes using the mapping of (2.41) of [2].

$$z = \sqrt{1 - \frac{\omega_p^2}{s^2}}, \text{real}(z) \geq 0 \quad (7)$$

The equations of (6) were solved using Matlab's `fsolve()` non-linear solver. Programming and debugging this approach was non-trivial; the interested reader is referred to the Matlab code. The initial values were found to be important in order to achieve convergence. The approach chosen was to start with frequency-scaled poles of a Bessel filter having the same order. The final solution was found to be robust to orders of 13 and larger which is greater than available in tables.

IV. FINITE TRANSMISSION ZEROS

Including finite-frequency transmission-zeros (often called loss-poles) was again based on the approach described in [2]. This approach is based on the transform:

$$w = \sqrt{1 + \frac{s^2}{\omega_s^2}}, \text{real}(w) \geq 0 \quad (8)$$

The procedure transforms the natural modes of the original denominator polynomial into the w domain, uses the transformed nodes to realize a polynomial in the w domain, selects the even part of this polynomial, finds the roots of the even terms, transforms them back into the s domain, and selects half of the resulting roots, so they are in the left-half plane. A Matlab routine realizing this procedure is:

```
function H = equalGDLossPoles(H1, ws, ap)
s2w = @(s) sqrt(1 + s.^2/ws.^2);
w2s = @(w) ws*sqrt(w.^2 - 1);

[z p k] = zpkdata(H1, 'vector');
p2 = -s2w(p);
```

```

ply = poly(p2);
plev = getEven(ply);
pE = roots(plev);
zs = w2s(pE);
zs = abs(imag(zs))*j;
for i=2:2:length(zs);
    zs(i) = -zs(i);
end
H = zpk(zs, p, 1);
H.k = H.k/ evalfr (H, 0);

```

Finite transmission zeros can be optionally added to both Bessel-Thomson filters and EQUI-GD filters

Except for some realization details, the preceding procedures represent Matlab realizations of existing methods; it's main contribution is that previously computer programs for most of these procedures were not currently available. The next section describes how to use filters based on these well-known linear-phase continuous-time filters as the basis to realize almost linear-phase digital filters with high accuracies; this is considered to be the major contribution of this paper.

V. DIGITAL FILTERS BASED ON LINEAR PHASE CONTINUOUS-TIME FILTERS

One of the most popular methods for realizing digital filters is based on the bilinear-z transform [3]. When using this method, s in the continuous-time filter is replaced using the substitution:

$$s \rightarrow \frac{2}{T} \frac{1 - z^{-1}}{1 + z^{-1}} \quad (9)$$

This procedure has desirable properties of preserving stability, and having a one-to-one mapping of magnitude frequency response, but due to the compression of stop-band frequencies (and pole frequencies), it does not work well for transforming IIR EQUI-GD filters; the linear-phase approximation is significantly degraded. This paper describes an iterative method based on adapting poles to restore the equiripple group-delay of the digital filter.

The procedure makes heavy use of a routine used for analyzing discrete-time transfer functions for their magnitude and phase responses, and derivatives, similar to what was presented for continuous-time transfer functions. The Matlab routine for this is :

```

function [lgH, phH, gdH, dLdW, dTdW] = AnlzDH(H, w)
[z, p, k] = zpkdata(H, 'vector');
npts = length(w);
onesVctr = ones(size(w));
nz = length(z);
np = length(p);
derivSum = zeros(size(w));
logSum = zeros(size(w));
ddrvSum = zeros(size(w));
eS = exp(j*w);
for l = 1:nz
    logSum = logSum + log((eS - z(l)));
    derivSum = derivSum + eS./(eS - z(l)*onesVctr);
    ddrvSum = ddrvSum + (eS.*z(l))./((eS - z(l)*onesVctr).^2);
end
for l = 1:np
    logSum = logSum - log((eS - p(l)));
    derivSum = derivSum - eS./(eS - p(l)*onesVctr);
    ddrvSum = ddrvSum - (eS.*p(l))./((eS - p(l)*onesVctr).^2);

```

```

end
phH = imag(logSum);
lgH = log(k) + real(logSum);
gdH = -real(derivSum);
% -real instead of imag because we didn't
% multiply by j when finding first order derivatives
dLdW = -imag(derivSum);
dTdW = -imag(ddrvSum);

```

The iterative procedure starts with the transformed poles of the continuous-time filter and does a minor frequency scaling on the poles to obtain an improved starting point. Using this starting point, the iterative procedure is: Analyze the discrete filter for its group-delay. Next find the extrema of the group-delay around the region that equiripple group-delay is expected. The extrema are found by analyzing for the zeros of the derivative of the group-delay with respect to w . The accuracy of the zero frequencies is improved by using interpolation. This process failed often due to glitches caused at finite loss-pole frequencies. These failures were eliminated by using Matlab's median filter function (medfilt1()). Once the extrema of the group-delay were found, an error vector was constructed by subtracting the extrema from the desired maximum and minimum group delays in the equiripple region. Next, a sensitivity matrix of the group delay derivatives, at the extrema frequencies with respect to the real and imaginary parts of the poles was derived. The error vector was multiplied by the inverse of this sensitivity matrix to obtain necessary pole changes to reduce the error vector. This procedure was repeated ten times for high order filters which resulted in equiripple group delays with excellent accuracies.

The Matlab code for the just the iterative portion of the procedure is:

```

for i = 1:10
    H = zpk(z, p, k);
    wz = findZeroCrS(H);
    Tz = p2T(H, wz);
    Y = setY2(Tz, gd1, gd2);
    sens = plSens(p, wz);
    dX = sens\Y;
    dP = rq2dp(dX);
    p = p - dP;
    T = p2T(H, w);
    plot(w,T)
end

```

The critical subroutine for finding the sensitivities of the group-delay with respect to pole real and imaginary parts is:

```

function sens = plSens(p, w)
w = w(:);
eS = exp(j*w);
onesVctr = ones(size(w));
np = length(p);
npts = length(w);
sens = zeros(npts, np*2);
for l = 1:np
    sens(:, 1) = real(eS./((eS - p(l)*onesVctr).^2));
    sens(:, 1+np) = -imag(eS./((eS - p(l)*onesVctr).^2));
end

```

The code for all of the Matlab routines will be available under an open-source license at www.github.com by the conference date.

VI. EXAMPLE: 11'TH ORDER DIGITAL FILTER WITH EQUIRIPPLE GROUP-DELAY

Most tables for continuous-time linear-phase filters cover orders up to 11; the Matlab routines described have been used for orders as high as 15 with high accuracy. The example being presented is an 11'th order filter intended for a 64-channel filter-bank application. The group delay of 10 iterations when adapting poles to correct the group-delay of the prototype digital filter (obtained after using the bilinear-z transform) is Shown in Fig. 1.

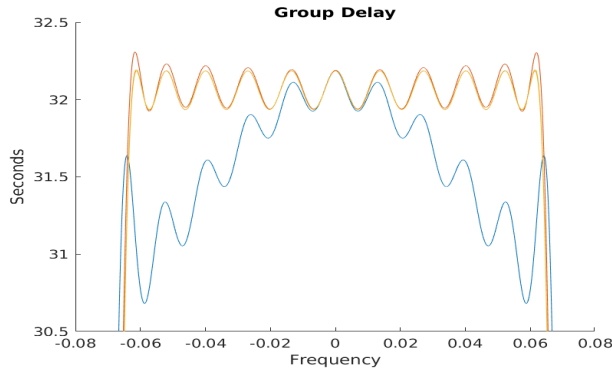


Fig. 1. Group delay of 11'th order digital filter during 10 iterations of adapting to eliminate deviations caused by bilinear-z transform.

It can be seen the algorithm converges quickly; after just a few iterations, the errors are not perceptible. The magnitude and phase responses along with a larger plot of the group delay of the filter are shown in Fig. 2.

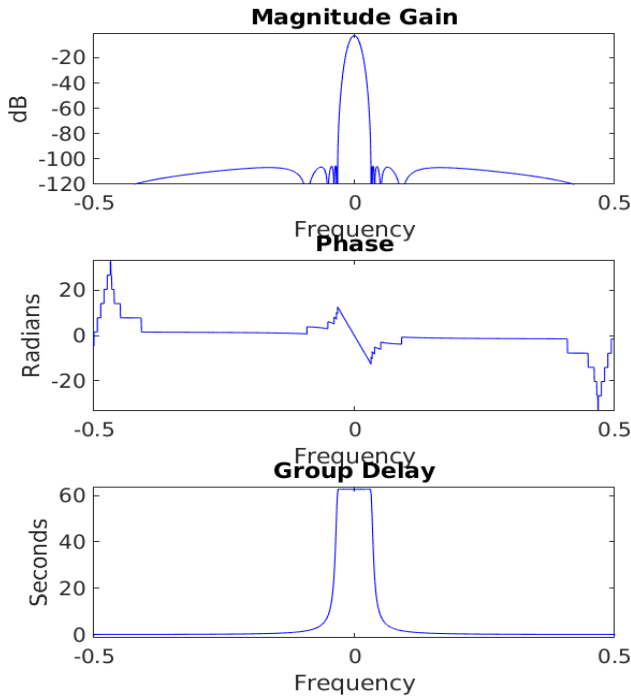


Fig. 2. Magnitude, phase, and group-delay of 11'th order digital filter.

The filter was used as a prototype filter for a 64-channel filter bank composed of 11'th order cascade complex filters using design procedures described in [4]. Shown in Fig. 3 are all 64 magnitude responses. The red line is the sum of all 64 channels. The ripple of the sum of the responses is 0.011dB. The closeness of this figure to 0dB is a figure of merit on the suitability of the filters for communication applications based on maximally decimated filter-banks. The figure of merit is good; we are currently investigating this application.

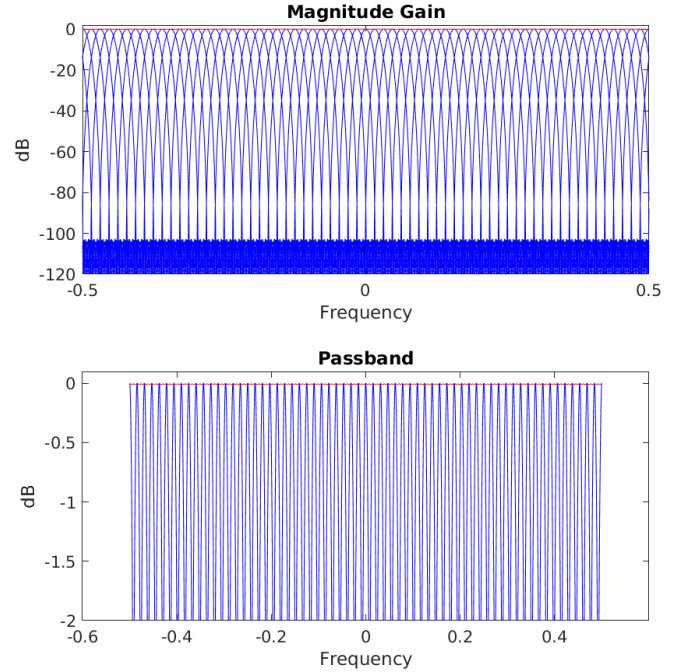


Fig. 3. Responses of cascade filters of 64-channel complex filter-bank based on equiripple prototype filter. Red line is sum of all 64 responses

VII. CONCLUSION

Design procedures for digital IIR filters having equiripple group delay and finite loss-poles have been presented. The routines can design high-order filters having approximately linear phase shift in the pass-band. An 11'th order example has been described. The example includes actual simulations of a 64-channel filter-bank realized using cascade filters based on an equiripple group-delay prototype filter.

REFERENCES

- [1] Introduction to Circuit Synthesis and Design, G.C. Temes and W. LaPatra, McGraw Hill, 1977.
- [2] Temes, G.C, "Approximation Theory," in G. C. Temes and S. K. Mitra (eds.), Modern Filter Theory and Design, Wiley, New York, 1973.
- [3] A.V. Oppenheim and R.W. Schaffer, Digital Signal Processing, Prentice-Hall, 1975.
- [4] K. Martin, Complex Cascade Digital Filters, *Intl. Symp. on Circuits and Systems*, pp. May 2019.
- [5] https://github.com/kwmartin/KM_ComplexFilterToolbox.
- [6] K. Martin, "Complex Signal Processing is Not Complex", *IEEE Trans. Circuits and Systems I*, Vol. 51, No. 9, pp. 1823-1836, Sept. 2004.